REDE DE ENSINO DOCTUM - CATAGUASES

ALLISON SOARES

LARISSA BATISTA

LUIZ HENRIQUE

MILTON FERNANDES

SISTEMAS DE INFORMAÇÃO – PROJETO INTEGRADOR IV

CATAGUASES

2026

MIGRAÇÃO E REESTRUTURAÇÃO DE BANCO DE DADOS RELACIONAL PARA ARQUITETURA MULTI-SCHEMA NO POSTGRESQL

1. INTRODUÇÃO

O presente trabalho aborda o desenvolvimento e a reestruturação da infraestrutura de banco de dados do sistema EcoTurismo, uma empresa fictícia voltada para o setor de turismo ecológico, responsável pelo gerenciamento de pacotes de viagem, reservas de clientes e conciliação de pagamentos.

Originalmente, o sistema operava em uma arquitetura legada baseada no SGBD MySQL, na qual todas as tabelas e dados estavam centralizados em uma única base sem divisões lógicas. À medida que o sistema evoluiu, identificou-se a necessidade de migrar para o PostgreSQL. A escolha pelo PostgreSQL justifica-se pela sua conformidade rigorosa com o padrão ACID, pela robustez no suporte a tipos customizados e validações via expressões regulares, e, prioritariamente, pela sua capacidade nativa de trabalhar com múltiplos schemas, garantindo maior isolamento de dados, modularidade e controle granular de permissões de acesso.

O objetivo do projeto é modernizar a camada de persistência da EcoTurismo, garantindo que os módulos administrativo, comercial e contábil operem de forma independente e segura.

2. METODOLOGIA

A primeira etapa do projeto concentrou-se na migração do banco de dados MySQL para o PostgreSQL e na reorganização de sua estrutura lógica.

2.1. Processo de Migração do MySQL para o PostgreSQL

O processo de conversão dos dados e do DDL (Data Definition Language) envolveu a adequação dos tipos de dados e recursos específicos de cada SGBD:

Chaves Primárias Autoincrementáveis: As colunas que utilizavam a propriedade AUTO_INCREMENT no MySQL foram convertidas para a sintaxe SERIAL no PostgreSQL.

Tipos de Dados Temporais e Numéricos: Colunas contendo valores monetários foram ajustadas para o tipo numeric(13,2) para evitar perdas de precisão decimal, e as datas foram padronizadas para o tipo date.

Criação de Tipos Enumerados (ENUM): Para a tabela de pagamentos, substituiu-se o uso de strings livres ou tabelas auxiliares pelo recurso nativo de tipos declarativos do PostgreSQL (CREATE TYPE contabil.forma AS ENUM ('PIX', 'BOLETO', 'CARTAO', 'A_VISTA')).

2.2. Separação da Arquitetura em Schemas

Para sanar o problema do acoplamento de dados, o banco foi particionado em três schemas lógicos distintos:

admin: Agrupa as entidades relacionadas à administração do sistema, controle de acessos, credenciais de usuários e permissões (admin.usuarios).

site: Módulo voltado para a aplicação web e e-commerce, englobando o catálogo de pacotes ecológicos (site.pacotes) e as solicitações de agendamento (site.reservas).

contabil: Módulo financeiro encarregado de registrar o histórico e as formas de pagamento das reservas efetuadas (contabil.pagamentos).

Essa segregação permite restringir o acesso das aplicações aos schemas estritamente necessários para o seu funcionamento, eliminando a exposição direta de dados contábeis ou administrativos para o módulo público de e-commerce.

2.3. Implementação de funções e triggers para automatização de reservas e pagamentos

Como complemento à organização dos schemas, foram implementadas duas funções em PL/pgSQL no schema contabil, associadas a triggers da tabela contabil.pagamentos. O objetivo é automatizar regras entre pagamentos e reservas usando as tabelas já existentes no EcoTurismo, sem recriar a estrutura do banco.

A função contabil.fn_validar_pagamento_reserva() é chamada pela trigger trg_validar_pagamento_reserva, executada antes de inserir um pagamento ou alterar o vínculo com uma reserva. A função consulta o status da reserva e guarda o resultado em uma variável. Se o status for Cancelada, RAISE EXCEPTION interrompe a operação e apresenta uma mensagem de erro; caso contrário, a função retorna a nova linha e permite que a operação prossiga.

A função contabil.fn_sincroniza_status_reserva() é chamada pela trigger trg_sincroniza_status_reserva, executada após INSERT, UPDATE ou DELETE em contabil.pagamentos, uma vez para cada linha afetada. TG_OP identifica a operação, enquanto NEW e OLD permitem acessar os dados novos e anteriores. Na inserção, uma reserva Pendente passa para Confirmada. Na exclusão, se a reserva estiver Confirmada e não restarem pagamentos associados, seu status volta para Pendente. Na alteração, a função verifica se o pagamento mudou de reserva, trata a reserva anterior quando fica sem pagamentos e confirma a nova reserva se estiver Pendente.

As triggers ficam vinculadas a contabil.pagamentos, mas as funções também podem atualizar o status em site.reservas, demonstrando a interação entre schemas. Na definição atual, qualquer UPDATE em contabil.pagamentos aciona a trigger de sincronização, inclusive alterações na data ou na forma de pagamento.

O funcionamento da inserção foi verificado em uma transação de teste: ao inserir um pagamento para uma reserva Pendente, uma consulta mostrou o status Confirmada. Em seguida, ROLLBACK desfez o pagamento de teste e a alteração do status, evitando manter os dados de demonstração.

Como limitação, a tabela contabil.pagamentos armazena a data, a forma e a reserva relacionada, mas não contém o valor pago nem o status de aprovação financeira. Assim, a regra considera a existência do registro de pagamento suficiente para confirmar a reserva; ela automatiza o status, mas não comprova, por si só, o recebimento do dinheiro.